Tecnologías de Registro Distribuido y Blockchain (BC)
Práctica 1 · Ejercicio 5

Contrato inteligente Crowdfunding

Plataforma de micromecenazgo «todo o nada» en Ethereum

Xoel Couto Vilas

1. Análisis y definición del escenario

1.1 Problema

Las plataformas de micromecenazgo tradicionales (tipo Kickstarter) funcionan con un modelo «todo o nada»: el creador de un proyecto fija un objetivo económico y un plazo; si se alcanza, cobra; si no, se devuelve el dinero a los donantes. Este modelo presenta tres problemas:

- **Custodia de los fondos:** el dinero lo guarda la plataforma, que debe ser de confianza.
- **Comisiones:** la plataforma cobra un porcentaje de lo recaudado.
- **Opacidad:** las reglas de cobro y devolución las aplica la plataforma y pueden cambiar o no cumplirse.

1.2 ¿Por qué necesita una blockchain pública?

Necesidad	Cómo la cubre Ethereum
Ninguna parte debe confiar en un intermediario	Los fondos quedan bloqueados en el contrato; solo se liberan según reglas públicas
Las reglas no deben poder cambiar tras el inicio	El código desplegado es inmutable
Cualquiera debe poder auditar el estado	Código, saldo y eventos son públicos y verificables
Pagos automáticos sin intermediarios	El contrato transfiere ETH por sí mismo cuando se cumplen las condiciones
Identidad de los participantes	Las direcciones (`address`) y `msg.sender` identifican quién actúa, sin registro previo

1.3 Solución propuesta

Un contrato inteligente `Crowdfunding` que gestiona **múltiples campañas** de micromecenazgo «todo o nada»:

- Cualquiera crea una campaña con un **objetivo** (en wei) y una **duración**.
- Los donantes aportan ETH mientras la campaña esté **activa**.
- Si al vencer el plazo se alcanzó el objetivo, **el creador retira los fondos**.
- Si no se alcanzó, **cada donante recupera exactamente lo que aportó**.

No existe propietario ni administrador del contrato: nadie puede tocar los fondos fuera de estas reglas.

2. Diseño

2.1 Caso de uso (lógica de negocio)

Ciclo de vida de una campaña. Las transiciones de `Activa` a `Exitosa` o `Fallida` ocurren **al vencer el plazo**, según lo recaudado frente al objetivo.

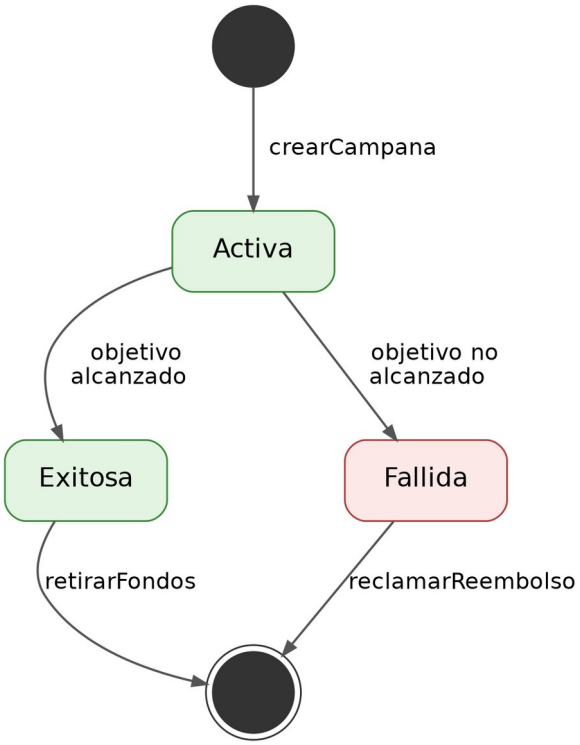


Figura 1. Diagrama de estados de una campaña

Estado	Operaciones permitidas
Activa	`contribuir` (las veces que se quiera hasta que venza el plazo)
Exitosa	`retirarFondos` (solo el creador, una única vez)
Fallida	`reclamarReembolso` (cada donante recupera lo suyo, una vez)

Los estados `Exitosa` y `Fallida` no se almacenan: se **calculan** a partir de la hora actual, lo recaudado y el objetivo (función `estado`). Así se ahorra una escritura en almacenamiento y se evita que el estado quede desfasado.

2.2 Usuarios del sistema (diagrama de casos de uso)

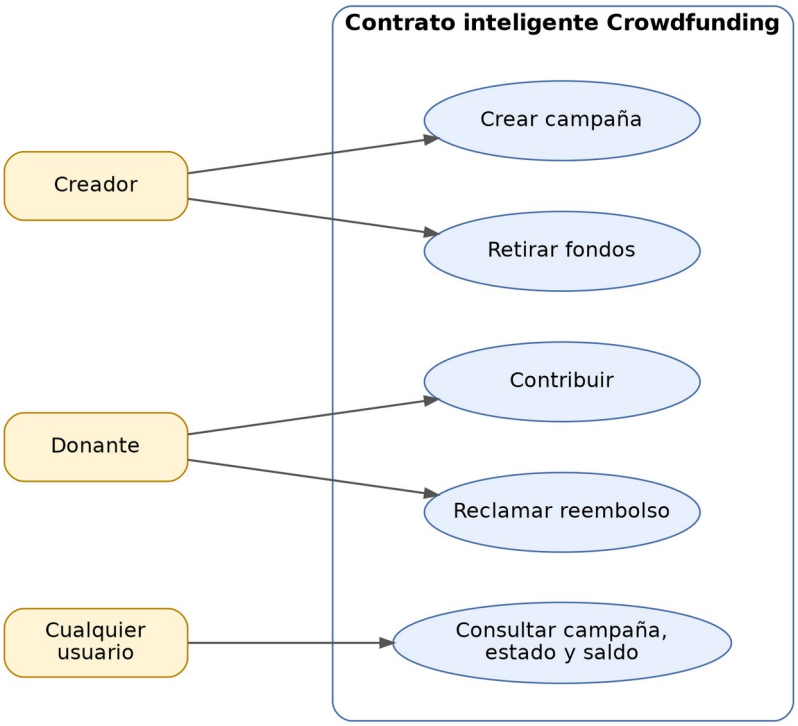


Figura 2. Diagrama de casos de uso

Actor	Casos de uso	Condiciones
Creador	Crear campaña	Objetivo > 0; duración entre 1 minuto y 365 días
	Retirar fondos	Solo el creador; plazo vencido; objetivo alcanzado; una única vez
Donante	Contribuir	Campaña existente y activa; aportación > 0
	Reclamar reembolso	Plazo vencido; objetivo **no** alcanzado; solo si aportó
Cualquier usuario	Consultar	Sin restricciones (funciones `view`)

> Un mismo `address` puede ser creador en una campaña y donante en otra.

2.3 Contenido del contrato inteligente

Datos

Elemento	Tipo	Descripción
`Estado`	`enum`	`Activa` (0), `Exitosa` (1), `Fallida` (2)
`Campana`	`struct`	`creador`, `objetivo`, `recaudado`, `fechaLimite`, `retirada`
`campanas`	`mapping(uint256 => Campana)`	Campañas por identificador (privado)
`aportaciones`	`mapping(uint256 => mapping(address => uint256))`	Wei aportado por cada donante a cada campaña (público)
`totalCampanas`	`uint256`	Contador de campañas; id de la última creada

Elemento	Tipo	Descripción
DURACION_MINIMA	constant uint40	1 minuto
DURACION_MAXIMA	constant uint40	365 días

Funciones

Función	Visibilidad	Descripción
crearCampana(_objetivo, _duracion)	external	Crea una campaña y devuelve su id
contribuir(_id)	external payable	Aporta ETH (msg.value) a una campaña activa
retirarFondos(_id)	external	El creador cobra lo recaudado
reclamarReembolso(_id)	external	El donante recupera su aportación
obtenerCampana(_id)	external view	Devuelve la struct completa
estado(_id)	external view	Devuelve Activa, Exitosa o Fallida
saldoContrato()	external view	Wei custodiados (address(this).balance)
_cargarCampana(_id)	private view	Lee la campaña y comprueba que existe
_objetivoAlcanzado(_c)	private pure	recaudado >= objetivo

Eventos

CampanaCreada, AportacionRealizada, FondosRetirados y ReembolsoRealizado. Permiten a un front-end (por ejemplo, Web3.js) reconstruir el histórico sin leer almacenamiento.

Reglas de operación

#	Regla	Error si se incumple
R1	El objetivo debe ser mayor que 0	ObjetivoInvalido
R2	La duración debe estar entre 1 minuto y 365 días	DuracionInvalida
R3	Solo se contribuye antes de la fecha límite	CampanaCerrada
R4	La aportación debe ser > 0 y caber en uint96	AportacionInvalida
R5	Solo el creador retira fondos	NoEsCreador
R6	No se retira ni se reembolsa antes del plazo	CampanaActiva
R7	Solo se retira si se alcanzó el objetivo	ObjetivoNoAlcanzado
R8	Los fondos se retiran una sola vez	YaRetirada
R9	Solo se reembolsa si **no** se alcanzó el objetivo	ObjetivoAlcanzado
R10	Solo reclama quien aportó, y una sola vez	SinAportacion
R11	La campaña debe existir	CampanaInexistente
R12	La transferencia de ETH debe completarse	TransferenciaFallida

3. Implementación

El código completo está en el archivo Crowdfunding.sol (Solidity ^0.8.30, licencia GPL-3.0).

3.1 Conocimientos de la práctica aplicados

Concepto de la práctica	Dónde se aplica
Version pragma y SPDX	Cabecera: //SPDX-License-Identifier: GPL-3.0 y pragma solidity ^0.8.30;
Constantes en mayúsculas	DURACION_MINIMA, DURACION_MAXIMA
struct	Campana, creada de forma posicional
enum	Estado
mapping anidado y msg.sender	aportaciones[_id][msg.sender]
Parámetros con guion bajo	_id, _objetivo, _duracion, _c
Funciones privadas con guion bajo	_cargarCampana, _objetivoAlcanzado
payable, msg.value, address(this).balance	contribuir, saldoContrato
view y pure	Consultas y _objetivoAlcanzado
memory en tipos por referencia	Campana memory c, Campana memory _c
Conversión explícita (typecasting)	uint40(block.timestamp), uint96(msg.value)
Eventos (event / emit)	Cuatro eventos, uno por acción relevante
Gestión de errores: revert y assert	Errores personalizados con revert; assert para un invariante
Estructuras de control if	Validaciones de cada función

3.2 Técnicas de optimización de gas

Técnica	Dónde	Efecto
Empaquetado de struct	Campana ocupa 2 slots (address + uint96 = 32 B; uint96 + uint40 + bool = 18 B) en vez de 5	Menos SSTORE/SLOAD, las operaciones más caras
Errores personalizados	error X() + revert X() en lugar de require(..., "texto")	Menos bytecode y menos gas al revertir
Lectura única a memory	Campana memory c = _cargarCampana(_id)	2 SLOAD en vez de uno por cada campo consultado
Escritura mínima	Solo se escribe el campo que cambia (recaudado o retirada)	Evita reescribir la struct entera
unchecked acotado	Contador totalCampanas y suma de aportaciones	Evita comprobaciones de overflow donde es imposible desbordar
Constantes	DURACION_MINIMA, DURACION_MAXIMA	No ocupan almacenamiento

Técnica	Dónde	Efecto
Estado derivado, no guardado	`estado()` se calcula al vuelo	Evita una escritura por cambio de estado
Eventos en vez de arrays	Histórico en logs	Los logs son mucho más baratos que el almacenamiento
Pago pull, sin bucles	Cada donante reclama lo suyo	Coste constante; sin riesgo de superar el límite de gas
`external` en los puntos de entrada	Funciones públicas del contrato	Evita copiar parámetros innecesariamente

3.3 Seguridad

- **Reentrancia:** patrón Checks-Effects-Interactions. `retirada = true` y `aportaciones[...] = 0` se escriben **antes** de enviar el ETH, por lo que una llamada reentrante encuentra el estado ya actualizado y revierte.
- **Pull over push:** el contrato nunca envía fondos a terceros en masa; cada uno retira lo suyo. Un donante que no pueda recibir ETH solo se perjudica a sí mismo.
- **Aritmética:** Solidity 0.8 revierte ante overflow. Las dos zonas `unchecked` están justificadas con comentarios en el código.
- **Cast seguro:** `msg.value` se valida contra `type(uint96).max` antes de convertirlo a `uint96`.
- **Invariante:** `assert(address(this).balance >= monto)` antes de retirar; si fallara, sería un bug del contrato.
- **Sin privilegios:** no hay `owner` ni función de emergencia que permita mover fondos.

4. Pruebas

4.1 Entorno y procedimiento

Se usa **Remix IDE** (remix.ethereum.org), compilador **0.8.30** y entorno **Remix VM**. Las campañas de prueba tienen una duración de **60 segundos** (el mínimo permitido), porque Remix VM no permite avanzar el reloj con facilidad: se espera a que venza el plazo.

1. Crear `contracts/Crowdfunding.sol` y pegar el código.
2. Solidity Compiler → versión 0.8.30 → **Compile**.
3. Deploy & Run → entorno Remix VM → **Deploy**.
4. Para cada prueba, elegir la cuenta (Account), el Value y la unidad (Wei/Ether), y pulsar el botón de la función.

5. Limitaciones y mejoras futuras

- **Marca de tiempo:** `block.timestamp` puede ser ligeramente manipulado por el productor del bloque (unos segundos). Es aceptable para plazos de horas o días, pero no para plazos de segundos.
- **Todo o nada:** no hay hitos ni retiradas parciales. Una mejora sería liberar los fondos por fases según el avance del proyecto.

- **Sin cancelación:** el creador no puede cancelar una campaña antes del plazo. Habría que decidir cómo se devuelve el dinero en ese caso.
- **Sin comisión:** el modelo no genera ingresos para quien despliega el contrato; se podría añadir un porcentaje configurable.
- **Receptores de ETH:** si un creador o donante es un contrato que rechaza ETH, su cobro o reembolso falla (`TransferenciaFallida`) y se puede reintentar sin pérdida de fondos.
- **Front-end:** una interfaz web (Web3.js) que escuche los eventos mostraría las campañas y el progreso en tiempo real.

6. Conclusiones

Se ha diseñado e implementado un contrato de micromecenazgo «todo o nada» que justifica el uso de una blockchain pública: elimina al intermediario que custodia el dinero, hace públicas e inmutables las reglas de cobro y devolución, y automatiza los pagos. El código aplica los conceptos de la práctica (`struct`, `enum`, `mapping`, eventos, `payable`, `view`/`pure`, gestión de errores) y técnicas de optimización de gas (empaquetado de almacenamiento, errores personalizados, lecturas únicas a memoria), manteniendo un coste constante por operación y siguiendo el patrón Checks-Effects-Interactions para evitar reentrancia.